

교과목 3 : 초거대언어모델(LLM)

단원 2 : 자연어 딥러닝

DAY5. 자연어 처리를 위한 어택션과 트랜스포머

목 차

1. 최신 인공지능 언어모델		3
2. AI 자연어 분석과 텍스트 생성		9
3. GPT		13
4. Attention과 Transformer 구현		16

1. 최신 인공지능 언어모델

현대 언어모델의 발전과 주요 특징

1. 언어모델의 진화 과정

1. 규칙 기반 시스템과 통계적 언어모델에서 출발하여 현재는 **Transformer 기반의 대규모 언어모델(LLM)**로 발전하였다.
2. 기존 모델의 문맥 이해 한계를 극복하여 보다 자연스럽게 사람과 유사한 문장을 생성할 수 있게 되었다.

2. Transformer 아키텍처

1. 문장 속 단어들 사이의 관계와 중요도를 계산하는 **Attention 메커니즘**을 사용한다.
2. 방대한 데이터를 미리 학습하는 **사전학습(Pre-training)** 방식을 적용하여 다양한 작업에서 높은 성능을 제공한다.

3. 주요 언어모델의 종류

GPT 모델

- 자기회귀(Autoregressive) 방식으로 다음 단어를 예측하여 문장을 생성한다.
- 챗봇과 대화형 AI 시스템에 주로 사용된다.

BERT 모델

- 양방향(Bidirectional) 문맥을 동시에 이해한다.
- 텍스트 분류
- 감성 분석
- 질의응답
- 자연어 이해(NLU)에 강점을 가진다.

멀티모달 모델

다음과 같은 모델들이 등장하였다.

- T5
- ELECTRA
- CLIP
- DALL-E

이러한 모델들은 텍스트뿐 아니라

- 이미지
- 영상
- 음성

등 다양한 데이터를 함께 처리할 수 있다.

4. 언어모델의 적용 분야

현대 언어모델은 다음과 같은 다양한 분야에서 활용된다.

- 기계번역
- 검색
- 문서요약
- 고객상담
- 법률 서비스
- 금융 서비스
- 코딩 보조
- 교육
- 창작 지원

언어모델 전이학습

전이학습(Transfer Learning)의 개념

전이학습은 한 작업에서 학습한 지식을 다른 관련 작업에 재사용하는 머신러닝 기법이다.
예를 들어,

- 영어를 잘하는 사람이
- 다른 외국어를 비교적 쉽게 배우는 원리와 비슷하다.

파인튜닝(Fine-Tuning)

사전학습된 모델에 새로운 데이터를 추가하여 목적에 맞게 다시 학습시키는 과정이다.

이를 통해 일반적인 언어 지식을

- 의료
- 법률
- 금융

등 특정 산업의 전문 지식으로 확장할 수 있다.

Hugging Face 활용

최신 AI 모델을 쉽게 사용할 수 있도록

Hugging Face Transformers 라이브러리를 제공한다.

지원 프레임워크는 다음과 같다.

- PyTorch
- TensorFlow

이를 통해 복잡한 모델도 몇 줄의 코드만으로 사용할 수 있다.

프롬프트 엔지니어링(Prompt Engineering)

프롬프트만 잘 작성해도 원하는 결과를 얻을 수 있다.

대표적인 방식은 다음과 같다.

- Zero-shot
- One-shot
- Few-shot

적절한 역할(Role)을 부여하거나 예시를 제공하면 더욱 정확한 결과를 생성할 수 있다.

전이학습의 장단점

장점

- 적은 양의 데이터만으로도 높은 성능을 얻을 수 있다.
- 모델 개발 시간이 크게 단축된다.
- 학습 비용을 절감할 수 있다.

단점

- 사전학습 데이터의 편향(Bias)이 그대로 전달될 수 있다.
- 원래 학습 데이터와 새로운 데이터의 차이가 크면 성능이 떨어질 수 있다.

대규모 언어모델(LLM)의 기업 도입

기업은 범용 언어모델을 그대로 사용하는 것이 아니라
사내

- 문서
- 상품 정보
- 고객 데이터

등을 추가하여 기업 전용 AI 시스템을 구축한다.

대표적인 모델은 다음과 같다.

- GPT-3
- GPT-4
- Claude
- PaLM

RAG(Retrieval-Augmented Generation)

LLM 은 존재하지 않는 정보를 생성하는 **환각(Hallucination)** 문제가 발생할 수 있다.

이를 해결하기 위해

외부 문서를 검색한 후

검색 결과를 바탕으로 답변을 생성하는 **RAG** 기법을 사용한다.

또한 내부 문서를

- 벡터(Vector)
- 임베딩(Embedding)

으로 저장한 **벡터 데이터베이스(Vector DB)** 와 결합하여 더욱 정확한 답변을 생성한다.

Seq2Seq 와 Attention

Seq2Seq 모델은

- 인코더(Encoder)
- 디코더(Decoder)

구조로 구성된다.

Attention 메커니즘은 긴 문장을 처리할 때 발생하는 정보 손실을 줄이기 위해 중요한 단어에 높은 가중치를 부여한다.

마치 오픈북 시험에서 필요한 부분만 찾아보듯 문맥에 맞는 정보를 집중적으로 활용하여 문장을 생성한다.

상황별 최적 모델 선택 전략

1. 간단한 텍스트 분류

다음과 같은 전통적인 방법이 적합하다.

- 단어 빈도(Bag of Words)
- TF-IDF

2. 문맥 이해가 중요한 경우

다음 모델들이 적합하다.

- RNN
- LSTM
- Attention 기반 Seq2Seq

3. 최고의 성능이 필요한 경우

최신 Transformer 기반 모델을 사용하는 것이 효과적이다.

대표적으로

- ChatGPT
- GPT 계열 모델

을 활용하면 복잡한 자연어 처리 작업에서도 높은 성능을 얻을 수 있다.

- 언어모델은 규칙 기반에서 Transformer 기반의 대규모 언어모델(LLM)로 발전하였다.
- GPT 는 문장 생성에, BERT 는 문장 이해와 분석에 강점을 가진다.
- 사전학습(Pre-training)과 파인튜닝(Fine-tuning)을 통해 적은 데이터로도 높은 성능을 얻을 수 있다.
- Hugging Face Transformers 는 PyTorch 와 TensorFlow 환경에서 최신 언어모델을 쉽게 사용할 수 있도록 지원한다.

- 프롬프트 엔지니어링은 AI 의 응답 품질을 높이는 핵심 기술이며, Zero-shot, One-shot, Few-shot 기법이 널리 사용된다.
- 기업에서는 GPT, Claude 등의 LLM 에 사내 데이터를 결합하고 RAG 와 벡터 데이터베이스를 활용하여 정확한 AI 서비스를 구축한다.
- 작업 목적에 따라 TF-IDF, RNN/LSTM, Transformer(ChatGPT) 등 적절한 모델을 선택하는 것이 중요하다.

2. AI 자연어 분석과 텍스트 생성

기존 Seq2Seq 모델의 한계와 개선의 필요성

1. Seq2Seq 모델의 개요 및 주요 기능

1. Seq2Seq(Sequence-to-Sequence) 모델은 기존의 언어 인식, 분류, 판단 기능을 넘어 **문장을 새롭게 생성하거나 요약하는 생성형 모델**이다.
2. 문서 요약, 기계 번역 등 다양한 자연어 생성 분야에서 뛰어난 성능을 보인다.

2. 기존 Seq2Seq 모델의 구조적 한계

(1) 고정 크기 벡터의 문제

- 입력 문장을 하나의 고정 크기 벡터로 압축하는 과정에서 중요한 정보가 손실될 수 있다.
- 특히 긴 문장의 경우 정보 병목(Information Bottleneck) 현상이 발생한다.

(2) 긴 문장의 의미 손실

- 문장이 길어질수록 앞부분의 정보가 점차 희석된다.
- 결과적으로 문장의 전체 의미 전달이 불완전해질 수 있다.

(3) 문맥 이해 부족

- 단어 하나만 보고 번역하거나 생성하기 때문에 주변 문맥을 충분히 고려하지 못한다.
- 예를 들어 동음이의어인 "밤"이 **Night**인지 **Chestnut**인지 구분하지 못하는 오류가 발생할 수 있다.

(4) 긴 대화 유지의 어려움

- 대화가 길어질수록 이전 대화 내용을 계속 유지하기 어렵다.
- 따라서 긴 문맥을 필요로 하는 대화에서 성능이 떨어질 수 있다.

3. Seq2Seq 모델의 주요 개선 방향

1. 잔차 연결(Residual Connection), 층 정규화(Layer Normalization), 빔 서치(Beam Search) 등을 도입하여 정보 병목을 줄이고 학습 안정성을 높인다.
2. 가장 중요한 개선 방법으로 **Attention 메커니즘**이 도입되었다.
 - 고정 크기 벡터 하나에만 의존하지 않고 필요한 정보에 집중할 수 있도록 개선하였다.

어텐션(Attention) 메커니즘의 이해

인간의 정보 처리 방식을 모방한 Attention

1. 어텐션 메커니즘의 기본 발상

(1) 사람은 회의에서 중요한 내용은 자세히 메모하고 중요하지 않은 내용은 간단히 기록한다. 즉, 모든 정보를 동일하게 처리하지 않는다.

(2) 예를 들어

- 점심 메뉴를 선택할 때와
- 결혼식 식당을 선택할 때

생각하는 기준이 서로 다르다.

즉, 상황에 따라 중요도를 다르게 판단한다.

(3) 문맥과 상황에 따라 생성되는 답변의 내용과 길이가 달라진다.

2. 어텐션 메커니즘의 구조와 동작 원리

(1) Query, Key, Value

Attention은 다음 세 가지 벡터를 이용한다.

- Query
- Key
- Value

이를 이용하여 현재 필요한 정보가 무엇인지 계산한다.

(2) 모든 정보를 동일하게 기억하려고 하지 않는다.

사람이 메모할 때 중요한 내용만 적는 것처럼 중요한 정보에 집중하여 정보 손실을 최소화한다.

(3) 기계 번역에서는 현재 단어 하나만 보는 것이 아니라 주변 단어 전체를 함께 고려하여 번역한다. 이것이 Seq2Seq 모델의 가장 큰 개선점이다.

(4) Attention 외에도

- 잔차 연결(Residual Connection)
- 층 정규화(Layer Normalization)
- 빔 서치(Beam Search) 등을 함께 사용하여 학습 성능을 높인다.

(5) 고정 길이 벡터에 의존하지 않고 중요한 정보를 선택적으로 참고할 수 있도록 Attention이 도입되었다.

Attention 도입의 효과

1. 긴 문장 처리 능력 향상

Attention은 장거리(Long-range) 의존성을 효과적으로 처리한다.
따라서 문장이 길어져도 중간 정보가 쉽게 사라지지 않는다.

2. 문맥 유지 능력 향상

동적인 Context(문맥)를 생성하여 전체 문장의 정보를 계속 유지한다.

그 결과

- 번역 품질
- 문장 생성 품질
- 문맥 이해 능력

이 크게 향상된다.

6페이지. 트랜스포머(Transformer) 기반 모델의 이해

멀티헤드 어텐션을 통한 Transformer의 구조적 차별점

1. Transformer 모델의 핵심 구조

(1) 기존 RNN과 LSTM은 이전 정보를 얼마나 유지할 것인지에 초점을 맞추었다.

반면 Transformer는 인코더와 디코더 구조를 Attention 중심으로 재구성한 새로운 아키텍처이다.

(2) Transformer는 단순히 Attention 하나를 추가한 모델이 아니다. 인코딩 과정에서 어떤 정보에 집중할지 어떤 정보를 덜 중요하게 볼지를 스스로 학습한다.

(3) 문맥에 따라 정보의 중요도를 조절하여 필요한 정보는 강조하고 덜 중요한 정보는 약화시키는 유연한 처리가 가능하다.

2. 멀티헤드 어텐션(Multi-Head Attention)

(1) Transformer는 하나의 Attention만 사용하는 것이 아니라 여러 개의 Attention Head를 동시에 사용한다. 이를 통해 다양한 관점에서 문장을 분석할 수 있다.

(2) 상황에 따라 Attention Head의 개수와 배치를 조절할 수 있다.

(3) 이미 구현된 Transformer 모듈을 활용할 수 있으며, 최적의 Head 수와 배치를 선택하는 것이 모델 성능 향상의 중요한 요소이다.

- Seq2Seq 모델은 문장 생성과 기계 번역에 뛰어나지만 고정 길이 벡터 때문에 긴 문장에서 정보 손실이 발생한다.
- 이러한 한계를 해결하기 위해 **Attention 메커니즘**이 도입되었으며, 중요한 단어에 집중하여 문맥을 효과적으로 유지할 수 있게 되었다.
- Attention은 Query, Key, Value 구조를 사용하여 현재 필요한 정보를 선택적으로 참조한다.
- Transformer는 RNN/LSTM 대신 Attention을 중심으로 설계된 모델이며, 여러 개의 Attention을 동시에 사용하는 **Multi-Head Attention**을 통해 문장의 다양한 의미 관계를 병렬적으로 학습한다.
- 이러한 구조 덕분에 번역, 문장 생성, 요약, 질의응답 등 다양한 자연어 처리 작업에서 높은 성능을 달성할 수 있다.

3. GPT

GPT 의 개념

- GPT 는 **언어 모델(Language Model)** 에 속하는 인공지능 모델이다.
- 대규모 텍스트 데이터를 이용하여 **사전 훈련(Pre-training)** 된 모델이다.
- 특히 **문장 생성(Text Generation)** 에 강점을 가진다.
- **일방향성(Unidirectional)** 구조를 사용한다.
 - 문장의 **앞에서 뒤 방향으로** 순차적으로 다음 단어를 예측한다.
- 대표적인 예로 **ChatGPT** 가 있다.
 - 이전까지 입력된 단어들을 기반으로 **다음에 올 가장 적절한 단어를 예측하여 문장을 생성하는** 알고리즘이다.

OpenAI

- OpenAI 는 GPT 를 지속적으로 발전시켜 더욱 뛰어난 모델을 출시하고 있다.
- 대표적인 발전 사항은 다음과 같다.
 - 이미지를 이해하는 **멀티모달(Multimodal)** 기능 지원
 - 더욱 향상된 **인식 능력**
 - 향상된 **추론 능력**
 - 높은 성능의 상용(유료) 서비스 제공

GPT 의 특징

1. 사전 훈련(Pre-training)

- 매우 큰 규모의 텍스트 코퍼스를 이용하여 학습한다.
- 문장의 의미를 이해하고 다음 단어를 예측하도록 학습된다.
- 이러한 과정에서 **언어 모델(Language Model)** 의 역할을 수행한다.

2. Transformer 아키텍처

- GPT 는 **Transformer** 구조를 기반으로 만들어졌다.
- Transformer 는 자연어 처리에서 매우 뛰어난 성능을 가진 딥러닝 구조이다.
- **Self-Attention(셀프 어텐션)** 메커니즘을 사용하여 문장 속 단어들의 관계를 효과적으로 학습한다.

3. 파인튜닝(Fine-Tuning)

- 사전 학습된 GPT 모델을 다양한 자연어 처리 문제에 맞게 추가 학습하여 사용할 수 있다.
- 대표적인 활용 분야는 다음과 같다.
 - 텍스트 분류(Text Classification)
 - 기계 번역(Machine Translation)
 - 질문 응답(Question Answering)

4. 자연어 생성

- GPT 는 다음과 같은 다양한 길이의 텍스트를 생성할 수 있다.
 - 문장
 - 문단
 - 긴 문서
- 입력된 문장의 일부를 바탕으로 **이후의 내용을 자동 생성**한다.

GPT 의 활용

자연어 생성

GPT 는 자연어를 이해하고 생성하는 능력을 바탕으로 다양한 분야에서 활용된다.

대표적인 활용 분야는 다음과 같다.

- 기계 번역
- 대화형 챗봇
- 텍스트 생성
- 문서 요약
- 질문 응답
- 자동 문장 생성

다양한 버전 제공

OpenAI 는 GPT 시리즈를 지속적으로 발전시키고 있다.

대표적인 모델은 다음과 같다.

- GPT-1
- GPT-2
- GPT-3
- GPT-4

각 버전은 이전 버전보다

- 더 많은 학습 데이터
- 더 큰 모델 크기
- 향상된 성능
- 더욱 뛰어난 추론 능력

을 제공한다.

GPT 에게 질문하기

- GPT 는 대규모 데이터를 학습하였기 때문에 사용자의 질문에 대해 **자연스럽고 구조화된 형태의 답변**을 생성할 수 있다.
- 사용자는 질문을 입력하면 GPT 가 학습한 지식을 바탕으로 적절한 답변을 생성한다.

(슬라이드에서는 ChatGPT 화면을 예시로 제시하여 "인공지능이 무엇인가?"라는 질문에 대한 답변 결과를 보여준다.)

Fine-Tuning 활용

기존 GPT 모델은 **파인튜닝(Fine-Tuning)** 을 수행하여 다양한 응용 프로그램에 적용할 수 있다.

예를 들면 다음과 같다.

- 고객 상담 챗봇
- 문서 요약 시스템
- 자동 번역기
- 질의응답 시스템
- 문장 생성 서비스
- 업무 자동화 시스템
- 코드 생성 및 코드 보조 시스템

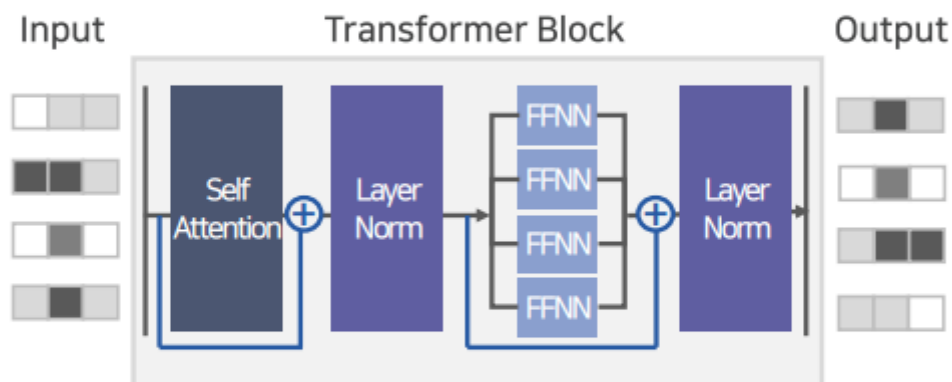
4. Attention과 Transformer 구현

1. Attention 과 Transformer 구현

1) Self-Attention과 Transformer

(1) Transformer Block 내부 구조

Self Attention, Normalization Layer#1, Feed Forward Neural Network, Normalization Layer#2 및 Residual Network(Skip Connection) 등으로 구성



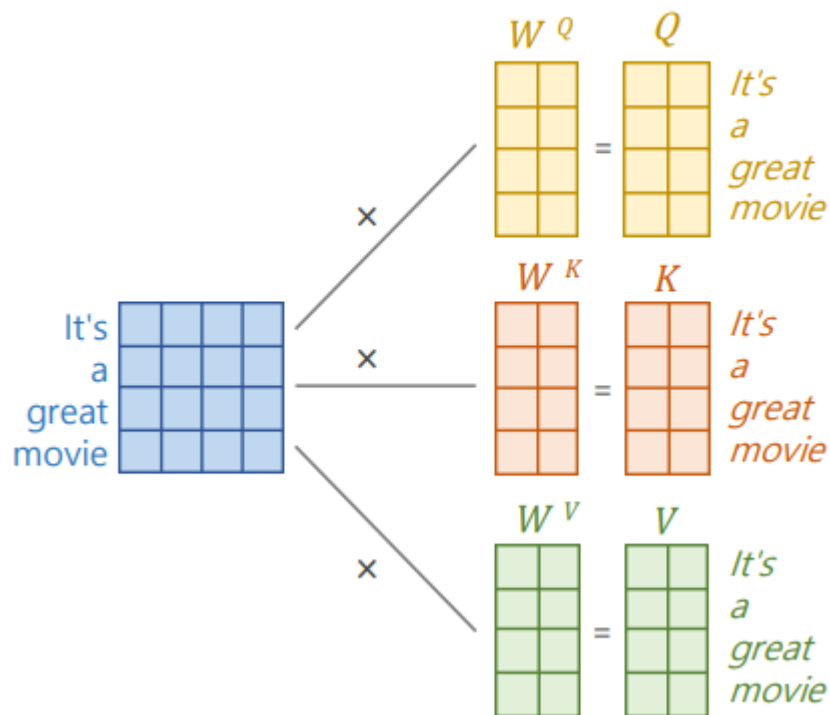
(2) Query, Key, Value 계산

Self-Attention의 첫 단계는 입력 문장에 대한 Query, Key, Value 계산

Input 문장의 512크기의 벡터와 학습할 Weight(WQ, WK, WV)를 곱하여 64크기의 Query, Key, Value 벡터 생성

Transformer 모델의 핵심 처리 과정을 나타낸 것이다. 입력 문장은 여러 개의 단어 벡터(토큰 임베딩)로 변환되어 Transformer Block 으로 입력된다. Transformer Block 은 Self-Attention → 잔차 연결(Residual Connection) → Layer Normalization → Feed Forward Neural Network(FFNN) → 잔차 연결 → Layer Normalization 순서로 데이터를 처리하여 더욱 의미 있는 문장 표현을 생성한다.

2) Self-Attention 구현



Self-Attention 의 첫 번째 단계인 **Query(Q)**, **Key(K)**, **Value(V)**를 생성하는 과정을 나타낸다. 입력 문장의 각 단어를 임베딩 벡터로 변환한 후, 각각 다른 학습 가중치 행렬(**WQ**, **WK**, **WV**)을 곱하여 Query, Key, Value 벡터를 생성한다. 이렇게 생성된 Q, K, V 는 단어 간의 관계와 중요도를 계산하는 데 사용된다.

2) Self-Attention 구현

(1) Query, Key, Value 계산

```
def forward(self, x: torch.Tensor) -> torch.Tensor:

    b, l, d = x.size()
    h = self.h

    queries = self.WQ(x).view(b, l, h, d).transpose(1, 2).contiguous().view(b * h, d)
    keys = self.WK(x).view(b, l, h, d).transpose(1, 2).contiguous().view(b * h, d)
    values = self.WV(x).view(b, l, h, d).transpose(1, 2).contiguous().view(b * h, d)

    w_prime = torch.bmm(queries, keys.transpose(1, 2)) / np.sqrt(d)
    w = F.softmax(w_prime, dim=-1)

    out = torch.bmm(w, values).view(b, h, l, d)

    out = out.transpose(1, 2).contiguous().view(b, l, h * d)

    return self.unifyheads(out)
```

Query에 Key의 Transpose한 행렬을 내적(Dot Product)

Query와 Key가 특정 문장에서 중요한 역할을 하고 있는 경우 Transformer는 이들 사이의 내적(Dot Product) 값을 크게 하는 방향으로 학습

내적 값이 커지면 해당 Query와 Key가 벡터 공간상 가까이에 있을 확률 높음

Self-Attention 의 첫 번째 단계인 **Query(Q)**, **Key(K)**, **Value(V)**를 생성하는 과정을 나타낸다. 입력 문장의 각 단어를 임베딩 벡터로 변환한 후, 각각 다른 학습 가중치 행렬(**WQ**, **WK**, **WV**)을 곱하여 Query, Key, Value 벡터를 생성한다. 이렇게 생성된 Q, K, V 는 단어 간의 관계와 중요도를 계산하는 데 사용된다.

1. 입력(Input)

예제 문장은 다음과 같다.

It's

a

great

movie

각 단어는 먼저 숫자로 표현된 **임베딩 벡터(Embedding Vector)** 로 변환된다.

예를 들어

It's → [0.12, 0.53, ...]

a → [0.28, 0.31, ...]

great → [0.75, 0.82, ...]

movie → [0.66, 0.14, ...]

이처럼 입력 문장은 하나의 **입력 행렬(Input Matrix)** 로 표현된다.

2. Query(Q) 생성

입력 행렬(X)에 **Query 가중치(WQ)** 를 곱하여 Query 를 만든다.

수식은 다음과 같다.

$$Q = X \times WQ$$

Query 는

"현재 단어가 어떤 정보를 찾고 싶은가?"

를 나타내는 벡터이다.

예를 들어

"great"라는 단어는

- 어떤 단어와 관련이 있는지
- 어떤 단어를 참고해야 하는지

질문(Query)을 생성한다.

3. Key(K) 생성

입력 행렬(X)에 **Key 가중치(WK)** 를 곱하여 Key 를 생성한다.

$$K = X \times WK$$

Key 는

"나는 어떤 정보를 가지고 있는가?"

를 나타내는 벡터이다.

Self-Attention 은 Query 와 Key 를 비교하여 단어 간의 관련성을 계산한다.

4. Value(V) 생성

입력 행렬(X)에 **Value 가중치(WV)** 를 곱하여 Value 를 생성한다.

$$V = X \times WV$$

Value 는

실제로 다음 단계에서 전달될 **의미 정보**를 담고 있는 벡터이다.

Attention Score 가 계산되면

중요한 Value 는 크게 반영되고

덜 중요한 Value 는 작게 반영된다.

5. 왜 세 개의 행렬을 사용하는가?

각 행렬은 서로 다른 역할을 수행한다.

행렬 역할

Query(Q) 현재 단어가 어떤 정보를 찾는지 표현

Key(K) 각 단어가 가지고 있는 특징 표현

Value(V) 실제 의미 정보를 저장

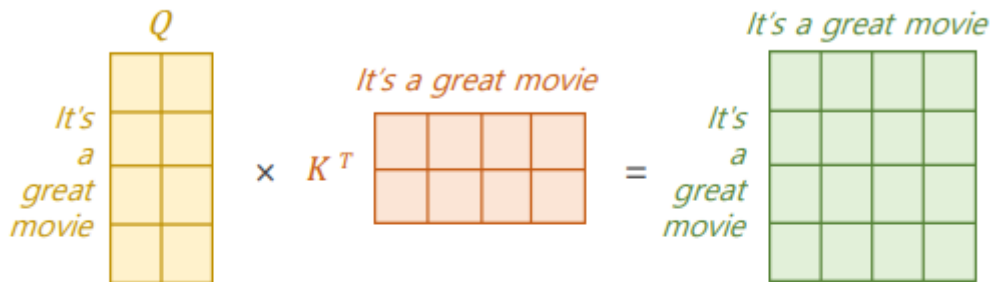
즉,

- Query 는 **질문**
- Key 는 **검색용 키**
- Value 는 **실제 내용**

이라고 생각하면 이해하기 쉽다.

2) Self-Attention 구현

(2) Self-Attention의 계산



```
def forward(self, x: torch.Tensor) -> torch.Tensor:

    b, l, d = x.size()
    h = self.h

    queries = self.WQ(x).view(b, l, h, d).transpose(1, 2).contiguous().view(b, h, l, d)
    keys = self.WK(x).view(b, l, h, d).transpose(1, 2).contiguous().view(b, h, l, d)
    values = self.WV(x).view(b, l, h, d).transpose(1, 2).contiguous().view(b, h, l, d)

    w_prime = torch.bmm(queries, keys.transpose(1, 2)) / np.sqrt(d)
    w = F.softmax(w_prime, dim=-1)

    out = torch.bmm(w, values).view(b, h, l, d)

    out = out.transpose(1, 2).contiguous().view(b, l, h * d)

    return self.unifyheads(out)
```

Self-Attention 에서 각 단어가 다른 단어와 얼마나 관련이 있는지 계산하는 과정을 나타낸다. 앞 단계에서 생성한 **Query(Q)** 와 **Key(K)** 를 이용하여 단어 간의 유사도(Attention Score)를 계산하고, 이를 바탕으로 **Value(V)** 의 중요한 정보를 선택하여 새로운 문장 표현을 만든다. 아래의 PyTorch 코드는 이러한 Self-Attention 계산을 실제로 구현한 예이다.

2) Self-Attention 구현

(2) Self-Attention의 계산

Key의 벡터 크기인 64의 root 값인 8로 나눈 후 softmax 함수 적용

Value와 내적(Dot Product)을 곱하여 Attention Value인 Z 계산

- $\sqrt{d_k}$ 로 나눈 이유는 Query와 Key의 내적 행렬의 분산을 축소하고 gradient vanishing 발생 방지

$$\text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) \times V = \text{Attention Value Matrix } \alpha$$

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Self-Attention의 최종 계산 과정을 보여준다. 앞 단계에서 생성한 **Query(Q)**, **Key(K)**, **Value(V)**를 이용하여 단어 간의 중요도를 계산하고, 그 결과를 이용해 문맥 정보를 반영한 새로운 단어 표현(Attention Value)을 생성한다. 이 과정을 **Scaled Dot-Product Attention**이라고 한다.

2) Self-Attention 구현

(2) Self-Attention의 계산

```
def forward(self, x: torch.Tensor) -> torch.Tensor:

    b, l, d = x.size()
    h = self.h

    queries = self.WQ(x).view(b, l, h, d).transpose(1, 2).contiguous().view(b * h, d)
    keys = self.WK(x).view(b, l, h, d).transpose(1, 2).contiguous().view(b * h, d)
    values = self.WV(x).view(b, l, h, d).transpose(1, 2).contiguous().view(b * h, d)

    w_prime = torch.bmm(queries, keys.transpose(1, 2)) / np.sqrt(d)
    w = F.softmax(w_prime, dim=-1)

    out = torch.bmm(w, values).view(b, h, l, d)

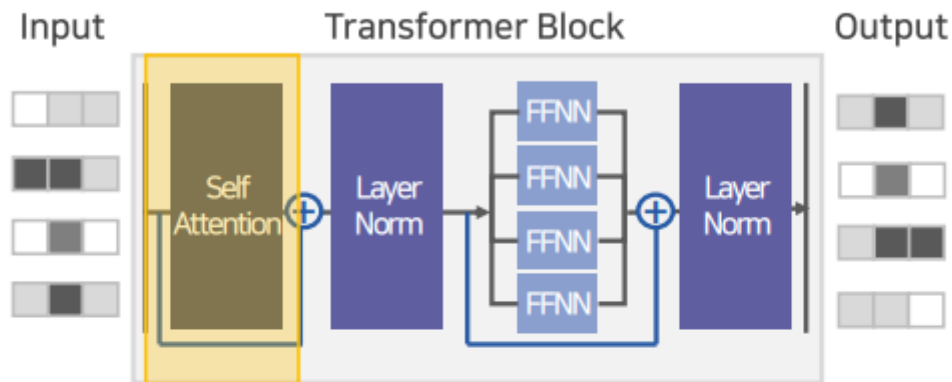
    out = out.transpose(1, 2).contiguous().view(b, l, h * d)

    return self.unifyheads(out)
```

Self-Attention 의 핵심 계산을 PyTorch 코드로 구현한 부분이다. 앞에서 생성한 Query(Q), Key(K), Value(V) 를 이용하여 단어 간의 중요도(Attention Weight)를 계산한 후, 이를 Value 에 적용하여 문맥(Context)을 반영한 새로운 단어 표현을 생성하는 과정이다.

3) Transformer 구현

(1) Self-Attention 추가



```
class TransformerBlock(nn.Module):  
    def __init__(self, d: int, heads: int=8, n_mlp: int=4):  
        super().__init__()  
  
        self.attention = SelfAttention(d, heads=heads)  
        self.norm1 = nn.LayerNorm(d)  
        self.norm2 = nn.LayerNorm(d)  
  
        self.ff = nn.Sequential(  
            nn.Linear(d, n_mlp*d),  
            nn.ReLU(),  
            nn.Linear(n_mlp*d, d)  
        )
```

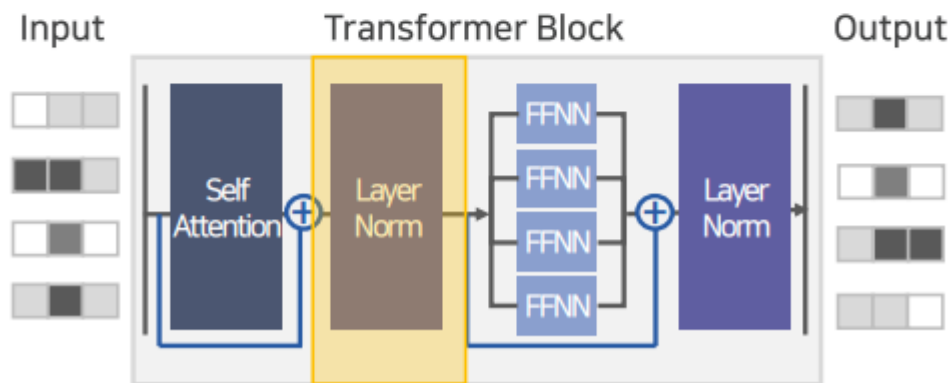
Transformer Block 을 구성하는 첫 번째 단계인 Self-Attention 계층을 추가하는 과정을 보여준다. 앞에서 구현한 Self-Attention 모듈을 Transformer Block 내부에 포함시키고, 이후 Layer Normalization 과 Feed Forward Neural Network(FFNN) 를 연결하여 하나의 Transformer Block 을 구성한다.

3) Transformer 구현

(1) Self-Attention 추가

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    x_prime = self.attention(x)
    x = self.norm1(x_prime + x)

    x_prime = self.ff(x)
    return self.norm2(x_prime + x)
```



Transformer Block에서 Self-Attention을 수행한 후 Residual Connection(잔차 연결)과 Layer Normalization을 적용하는 과정을 보여준다. 입력 문장은 먼저 Self-Attention을 통해 문맥 정보를 반영한 새로운 표현으로 변환되고, 원래 입력과 더한 후 Layer Normalization을 수행한다. 이후 Feed Forward Neural Network(FFNN)를 거친 결과에도 동일한 과정을 반복하여 최종 출력이 생성된다.

3) Transformer 구현

(1) Self-Attention 추가

```
class TransformerBlock(nn.Module):
    def __init__(self, d: int, heads: int=8, n_mlp: int=4):
        super().__init__()

        self.attention = SelfAttention(d, heads=heads)
        self.norm1 = nn.LayerNorm(d)
        self.norm2 = nn.LayerNorm(d)

        self.ff = nn.Sequential(
            nn.Linear(d, n_mlp*d),
            nn.ReLU(),
            nn.Linear(n_mlp*d, d)
        )
```

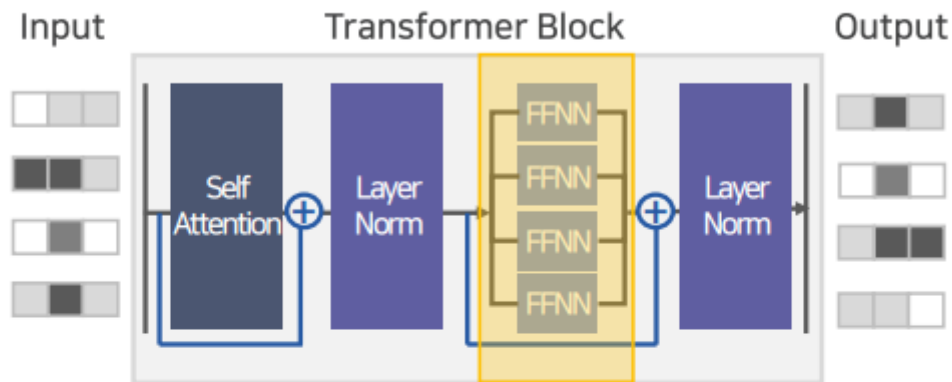
```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    x_prime = self.attention(x)
    x = self.norm1(x_prime + x)

    x_prime = self.ff(x)
    return self.norm2(x_prime + x)
```

Transformer Block 에서 첫 번째 Layer Normalization(LayerNorm)을 구현하는 과정을 보여준다. Self-Attention 연산이 끝난 후 **Residual Connection(잔차 연결)** 을 수행하고, 그 결과를 **Layer Normalization** 에 입력하여 데이터 분포를 안정화한다. 이 과정은 Transformer 모델이 깊어져도 안정적으로 학습할 수 있도록 하는 핵심 구성 요소이다.

3) Transformer 구현

(1) Self-Attention 추가



```
class TransformerBlock(nn.Module):
    def __init__(self, d: int, heads: int=8, n_mlp: int=4):
        super().__init__()

        self.attention = SelfAttention(d, heads=heads)
        self.norm1 = nn.LayerNorm(d)
        self.norm2 = nn.LayerNorm(d)

        self.ff = nn.Sequential(
            nn.Linear(d, n_mlp*d),
            nn.ReLU(),
            nn.Linear(n_mlp*d, d)
        )
```

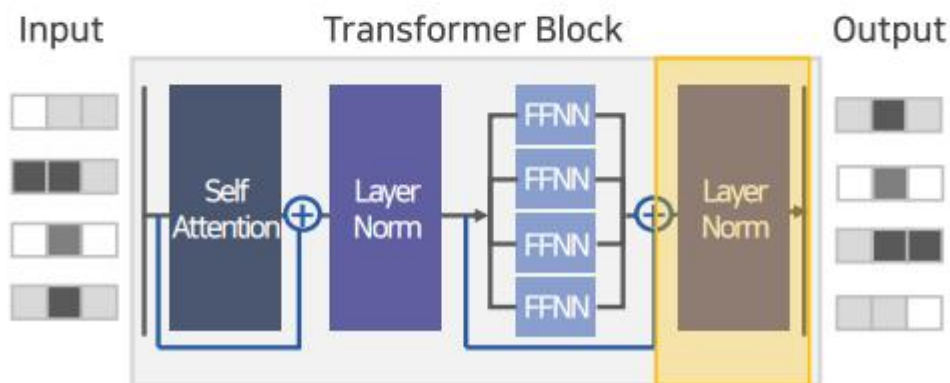
Transformer Block 의 두 번째 핵심 구성 요소인 Feed Forward Neural Network(FFNN) 를 보여준다. Self-Attention 이 문장 속 단어 간의 관계와 문맥(Context) 을 학습하는 역할을 한다면, FFNN 은 각 단어 벡터의 특징을 더욱 풍부하게 변환하고 표현력을 높이는 역할을 한다. 모든 단어는 FFNN 을 동일한 가중치(Shared Weights) 로 독립적으로 처리한다.

3) Transformer 구현

(1) Self-Attention 추가

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    x_prime = self.attention(x)
    x = self.norm1(x_prime + x)

    x_prime = self.ff(x)
    return self.norm2(x_prime + x)
```



Transformer Block 의 마지막 처리 과정을 보여준다. 첫 번째 **Self-Attention** → **Residual Connection** → **Layer Normalization** 을 수행한 후, **Feed Forward Neural Network(FFNN)** 를 거쳐 다시 **Residual Connection** 과 **Layer Normalization** 을 적용하여 최종 출력을 생성한다. 이 과정은 Transformer Block 을 완성하는 마지막 단계이며, 다음 Transformer Block 또는 출력층으로 전달될 최종 특징 벡터를 만든다.

3) Transformer 구현

(1) Self-Attention 추가

```
class TransformerBlock(nn.Module):
    def __init__(self, d: int, heads: int=8, n_mlp: int=4):
        super().__init__()

        self.attention = SelfAttention(d, heads=heads)
        self.norm1 = nn.LayerNorm(d)
        self.norm2 = nn.LayerNorm(d)

        self.ff = nn.Sequential(
            nn.Linear(d, n_mlp*d),
            nn.ReLU(),
            nn.Linear(n_mlp*d, d)
        )
```

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    x_prime = self.attention(x)
    x = self.norm1(x_prime + x)

    x_prime = self.ff(x)
    return self.norm2(x_prime + x)
```

Transformer Block 의 마지막 단계인 두 번째 Layer Normalization(LayerNorm) 을 구현하는 과정을 보여준다. Feed Forward Neural Network(FFNN)를 통과한 결과에 **Residual Connection(잔차 연결)** 을 적용한 후, **Layer Normalization** 을 수행하여 최종 출력을 생성한다. 이 출력은 다음 Transformer Block 의 입력 또는 모델의 최종 출력으로 전달된다.

2. Transformer 감성분석 적용

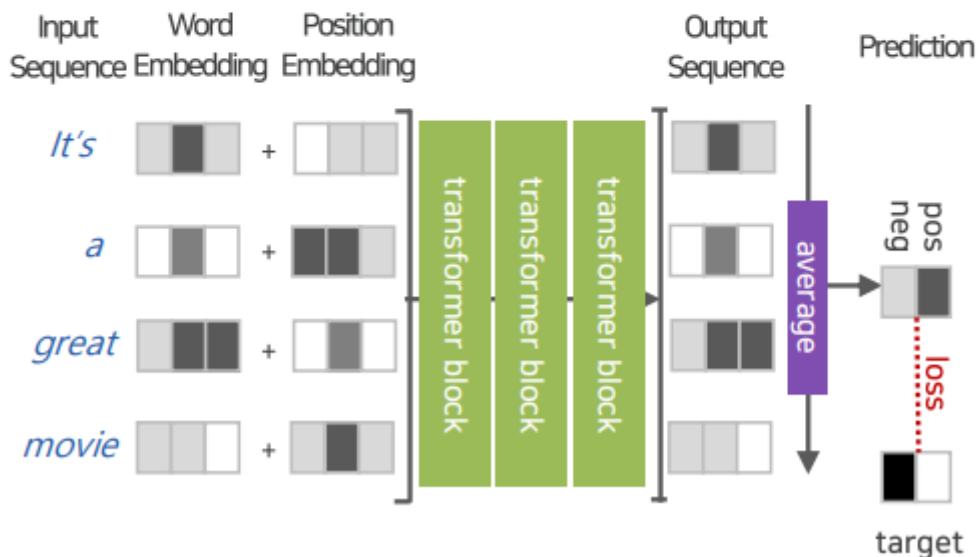
1) Transformer 적용 방법

(1) Transformer 전체 구조

Word embedding과 Positional Embedding 필요

Transformer를 멀티 Layer로 적용

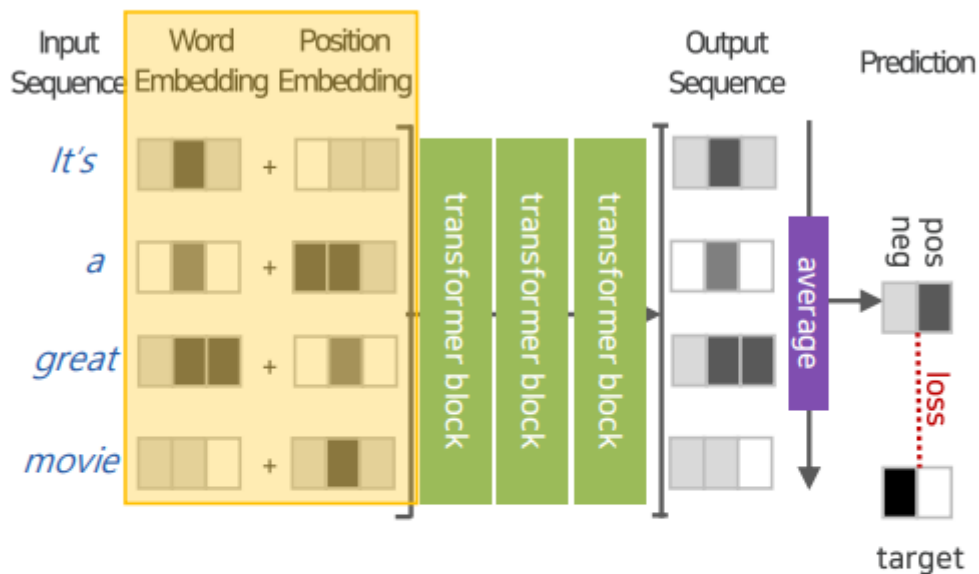
긍정/부정 학습 및 예측을 위한 분류기 추가



Transformer 를 문장 분류(감성 분석) 문제에 적용하는 전체 과정을 보여준다. 입력 문장은 **Word Embedding** 과 **Positional Embedding** 을 결합한 후 여러 개의 **Transformer Block** 을 통과한다. 이후 출력 벡터를 하나의 대표 벡터로 요약(Average Pooling)하고, **분류기(Classifier)** 를 통해 긍정(Positive) 또는 부정(Negative)을 예측한다. 마지막으로 예측 결과와 실제 정답을 비교하여 **손실(Loss)** 을 계산하고 모델을 학습한다.

2) Word Embedding과 Positional Embedding Layer

(1) Word Embedding과 Positional Embedding Layer 추가



(2) `__init__`

```
self.token_emb = nn.Embedding(num_tokens, d)
self.pos_emb = nn.Embedding(max_seq_len, d)
```

(3) `__forward__`

```
tokens = self.token_emb(x)
positions = self.pos_emb(torch.arange(1).to(self.device)).expand(b, 1,
embeddings = tokens + positions
```

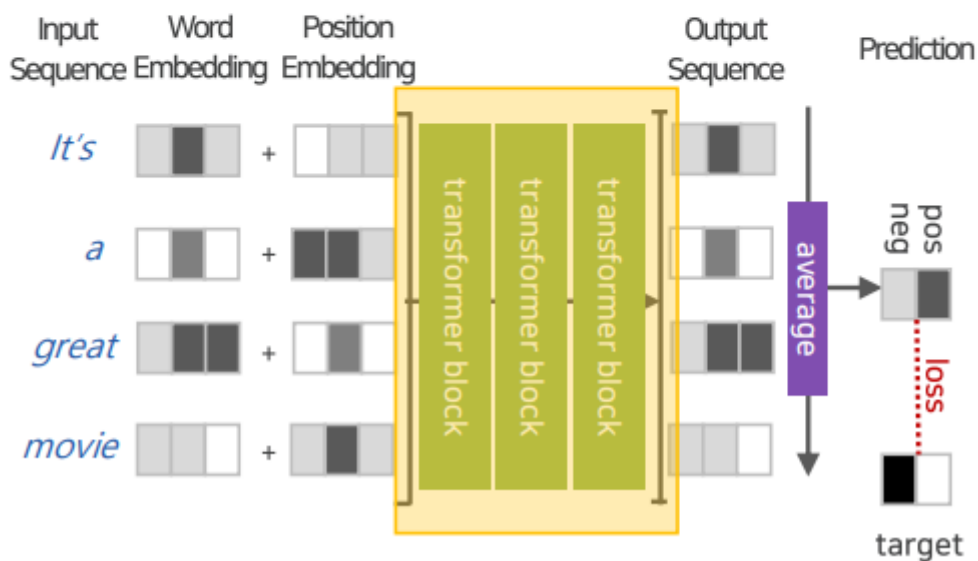
Transformer 모델의 입력 단계를 보여준다. Transformer 는 문장을 바로 입력받지 않고, 먼저 **Word Embedding** 과 **Positional Embedding** 을 생성한 후 두 벡터를 더하여 최종 입력 벡터를 만든다. 이 벡터가 여러 개의 **Transformer Block** 을 통과하면서 문맥을 학습하고, 마지막에는 문장 분류 또는 문장 생성에 사용된다.

2) Word Embedding과 Positional Embedding Layer

(4) Positional Embedding을 추가하는 이유?

- RNN + Attention
 - RNN 제거 후 Attention만으로 Transformer 구현
- RNN이 제거되어 토큰의 별도 순서 지정 필요

(5) Transformer Block 추가



Transformer 모델에서 Positional Embedding 이 필요한 이유와 여러 개의 Transformer Block 을 사용하는 이유를 설명한다. Transformer 는 RNN 과 달리 문장을 순차적으로 처리하지 않고 모든 단어를 동시에 병렬 처리한다. 따라서 단어의 순서 정보를 별도로 알려주기 위해 Positional Embedding 을 사용하며, 이후 여러 개의 Transformer Block 을 반복적으로 통과하면서 문장의 의미를 점차 정교하게 학습한다.

2) Word Embedding과 Positional Embedding Layer

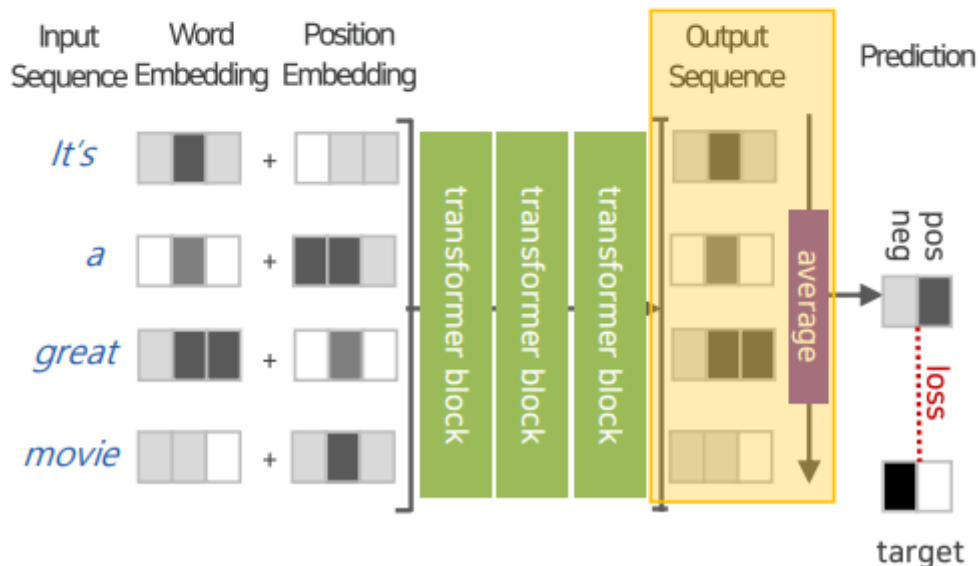
(6) __init__

```
self.transformer_blocks = nn.Sequential(  
    *[TransformerBlock(d=d, heads=heads) for _ in range(depth)]  
)
```

(7) __forward__

```
out = self.transformer_blocks(embeddings)
```

(8) Classification Layer 추가



Transformer 모델의 마지막 구성 단계를 보여준다. Word Embedding 과 Positional Embedding 을 결합한 입력을 여러 개의 **Transformer Block** 에 반복적으로 통과시켜 문맥 정보를 학습한 후, **Classification Layer**(분류층) 를 추가하여 문장의 클래스를 예측한다. 감성 분석에서는 최종적으로 **긍정(Positive)** 또는 **부정(Negative)** 를 예측하며, 예측 결과와 실제 정답(Target)을 비교하여 **Loss** 를 계산하고 모델을 학습한다.

2) Word Embedding과 Positional Embedding Layer

(9) `__init__`

```
self.classification = nn.Linear(d, num_classes)
```

(10) `__forward__`

```
out = out.mean(dim=1)
out = self.classification(out)
```

Transformer 모델의 마지막 단계인 **Classification Layer(분류층)** 를 구현하는 과정을 보여준다. 여러 개의 Transformer Block 을 통과한 **Output Sequence** 를 하나의 문장 벡터로 요약한 후, **완전연결층(Linear Layer)** 에 입력하여 최종 클래스를 예측한다. 감성 분석에서는 일반적으로 **긍정(Positive)** 또는 **부정(Negative)** 를 출력하며, 분류 결과를 이용하여 손실(Loss)을 계산하고 모델을 학습한다.

- `self.classification = nn.Linear(d, num_classes)`는 **Transformer** 의 최종 출력 벡터를 원하는 클래스 개수로 변환하는 **출력층(Classification Layer)** 을 생성하는 코드이다.
- `out.mean(dim=1)`은 **Output Sequence** 에 있는 모든 단어 벡터의 평균을 계산하여 문장 전체를 대표하는 하나의 벡터(Sentence Vector)를 생성하는 **Average Pooling** 연산이다.
- `out = self.classification(out)`는 문장 벡터를 **완전연결층(Linear Layer)** 에 입력하여 각 클래스의 점수(Class Score)를 계산하며, 이후 Softmax 를 적용하면 클래스별 확률을 얻을 수 있다.
- 감성 분석에서는 일반적으로 **긍정(Positive)** 과 **부정(Negative)** 의 두 클래스에 대한 점수를 계산하고, 가장 높은 확률을 가진 클래스를 최종 예측 결과로 선택한다.
- 이 과정은 Transformer 모델에서 **문맥 정보를 학습하는 부분과 실제 분류를 수행하는 부분을 연결하는 마지막 단계**이며, 계산된 예측 결과는 실제 정답(Target)과 비교되어 손실(Loss)을 계산하고, 역전파(Backpropagation)를 통해 모델을 학습시키는 데 사용된다.